LAB IMPLEMENTATION EXPERIMENT 4

Experiment 4: Implementing a Generative Adversarial Network (GAN) for Image Generation

Generative AI Laboratory (CO3)

Date: 22 October 2026

Aim

To implement a Generative Adversarial Network (DCGAN) that generates realistic handwritten-digit images from random noise, to train it with a custom adversarial loop in TensorFlow, and to evaluate the results.

Theory

A **Generative Adversarial Network** (Goodfellow et al., 2014) consists of two networks trained against each other:

- the **generator** $G(z)$ maps random noise $z \sim \mathcal{N}(0, I)$ to a synthetic sample;
- the **discriminator** $D(x)$ estimates the probability that x is real rather than generated.

They play the minimax game

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))].$$

At the optimum the generator distribution equals the data distribution and $D(x) = \frac{1}{2}$. In practice the generator minimises the *non-saturating* loss $-\log D(G(z))$, which gives stronger gradients early in training.

A **DCGAN** uses convolutional layers: strided transposed convolutions upsample in the generator, strided convolutions downsample in the discriminator, batch-normalisation and LeakyReLU stabilise training, and the generator ends with tanh so images live in $[-1, 1]$. Tricks used here: Adam with $\beta_1 = 0.5$, one-sided label smoothing (real label 0.9) and dropout in the discriminator. Typical failure modes are *mode collapse* (the generator produces few distinct outputs) and an unbalanced discriminator.

Steps

1. Load MNIST and scale the pixels to $[-1, 1]$; create a shuffled, batched `tf.data` pipeline.
2. Define the generator (Dense $\rightarrow$ 2 transposed-convolution blocks $\rightarrow$ tanh conv) and the discriminator (2 strided-convolution blocks $\rightarrow$ 1 logit).
3. Define the binary cross-entropy losses and two Adam optimisers.
4. Write a `tf.function` training step that updates both networks with `GradientTape`.
5. Train for 15 epochs while logging the losses and discriminator accuracy, saving samples from

a fixed noise batch.

6. Plot the loss curves and the evolution of the generated digits.
7. Generate a final grid of 100 digits and interpolate between two latent vectors.
8. Evaluate the generated digits with an independent MNIST classifier and check class diversity.

Code

```
import os, time
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
import numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
keras.utils.set_random_seed(7)
LATENT, BATCH, EPOCHS = 100, 128, 15
print("TensorFlow", tf.__version__, "| latent dim", LATENT, "| batch", BATCH, "| epochs", EPOCHS)

(x_train, _), _ = keras.datasets.mnist.load_data()
x_train = ((x_train.astype("float32") - 127.5) / 127.5)[..., None]
ds = tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000, seed=0).batch(BATCH, drop_remainder=True).prefetch(2)
print("training images:", x_train.shape, "range [%.1f, %.1f]" % (x_train.min(), x_train.max()))
print("steps per epoch:", len(ds))

def make_generator():
    return keras.Sequential([
        layers.Input((LATENT,)),
        layers.Dense(7 * 7 * 128, use_bias=False), layers.BatchNormalization(), layers.LeakyReLU(0.2),
        layers.Reshape((7, 7, 128)),
        layers.Conv2DTranspose(64, 5, strides=2, padding="same", use_bias=False),  # 14x14
        layers.BatchNormalization(), layers.LeakyReLU(0.2),
        layers.Conv2DTranspose(32, 5, strides=2, padding="same", use_bias=False),  # 28x28
        layers.BatchNormalization(), layers.LeakyReLU(0.2),
        layers.Conv2D(1, 5, padding="same", activation="tanh"),
    ], name="generator")

def make_discriminator():
    return keras.Sequential([
        layers.Input((28, 28, 1)),
        layers.Conv2D(32, 5, strides=2, padding="same"), layers.LeakyReLU(0.2), layers.Dropout(0.3),
        layers.Conv2D(64, 5, strides=2, padding="same"), layers.LeakyReLU(0.2), layers.Dropout(0.3),
        layers.Flatten(), layers.Dense(1),
    ], name="discriminator")

G, D = make_generator(), make_discriminator()
G.summary(print_fn=print); D.summary(print_fn=print)
z = tf.random.normal((1, LATENT))
```

```
print("\nuntrained G(z) shape:", G(z, training=False).shape, "| D(G(z)) logit:", float(D(G(z, training=False))[0, 0]))

bce = keras.losses.BinaryCrossentropy(from_logits=True)
g_opt = keras.optimizers.Adam(2e-4, beta_1=0.5)
d_opt = keras.optimizers.Adam(2e-4, beta_1=0.5)

@tf.function
def train_step(real):
    z = tf.random.normal((tf.shape(real)[0], LATENT))
    with tf.GradientTape() as gt, tf.GradientTape() as dt:
        fake = G(z, training=True)
        real_logit, fake_logit = D(real, training=True), D(fake, training=True)
        d_loss = bce(tf.ones_like(real_logit) * 0.9, real_logit) + bce(tf.zeros_like(fake_logit), fake_logit)
        g_loss = bce(tf.ones_like(fake_logit), fake_logit)
    g_opt.apply_gradients(zip(gt.gradient(g_loss, G.trainable_variables), G.trainable_variables))
    d_opt.apply_gradients(zip(dt.gradient(d_loss, D.trainable_variables), D.trainable_variables))
    d_acc = 0.5 * (tf.reduce_mean(tf.cast(real_logit > 0, tf.float32)) + tf.reduce_mean(tf.cast(fake_logit < 0, tf.float32)))
    return g_loss, d_loss, d_acc
print("train_step compiled with tf.function")

fixed_z = tf.random.normal((64, LATENT), seed=123)
snapshots, history = {}, []
t0 = time.time()
for epoch in range(1, EPOCHS + 1):
    gl, dl, da = [], [], []
    for batch in ds:
        g, d, a = train_step(batch)
        gl.append(float(g)); dl.append(float(d)); da.append(float(a))
    history.append((np.mean(gl), np.mean(dl), np.mean(da)))
    if epoch in (1, 2, 3, 5, 8, 12, EPOCHS):
        snapshots[epoch] = G(fixed_z, training=False).numpy()
    print(f"epoch {epoch:2d}/{EPOCHS}  g_loss {history[-1][0]:.3f}  d_loss {history[-1][1]:.3f}  "
          f"d_acc {history[-1][2]:.2f}  ({time.time() - t0:.0f}s)")

h = np.array(history)
fig, ax = plt.subplots(1, 2, figsize=(10, 3.3))
ax[0].plot(range(1, EPOCHS + 1), h[:, 0], label="generator"); ax[0].plot(range(1, EPOCHS + 1), h[:, 1], label="discriminator")
ax[0].set(title="GAN losses", xlabel="epoch"); ax[0].legend()
ax[1].plot(range(1, EPOCHS + 1), h[:, 2], color="tab:green"); ax[1].axhline(0.5, ls="--", c="gray")
ax[1].set(title="Discriminator accuracy (0.5 = cannot tell real from fake)", xlabel="epoch", ylim=(0.3, 1))
ax[1].title.set_fontsize(9)
plt.tight_layout(); plt.show()

def grid(imgs, n=8):
    imgs = (imgs[: n * n, ..., 0] + 1) / 2
    return np.concatenate([np.concatenate(imgs[r * n:(r + 1) * n], axis=1) for r in range(n)], axis=0)
eps = sorted(snapshots)[::1]
sel = [eps[0], eps[2], eps[4], eps[-1]]
```

```
fig, axes = plt.subplots(1, 4, figsize=(13, 3.6))
for a, e in zip(axes, sel):
    a.imshow(grid(snapshots[e], 6), cmap="gray"); a.set_title(f"after epoch {e}");
    a.axis("off")
plt.tight_layout(); plt.show()

final = G(tf.random.normal((100, LATENT), seed=2024), training=False).numpy()
plt.figure(figsize=(6, 6)); plt.imshow(grid(final, 10), cmap="gray"); plt.axis("off
    ")
plt.title(f"100 digits generated by the DCGAN after {EPOCHS} epochs"); plt.show()

z1, z2 = tf.random.normal((1, LATENT), seed=11), tf.random.normal((1, LATENT), seed
    =12)
alphas = np.linspace(0, 1, 10)
zs = tf.concat([(1 - a) * z1 + a * z2 for a in alphas], axis=0)
interp = (G(zs, training=False).numpy()[..., 0] + 1) / 2
fig, axes = plt.subplots(1, 10, figsize=(13, 1.8))
for a, im, al in zip(axes, interp, alphas):
    a.imshow(im, cmap="gray"); a.set_title(f"{al:.1f}", fontsize=8); a.axis("off")
plt.suptitle("Smooth transition in latent space (z1 -> z2)", y=1.05); plt.show()

(xc, yc), (xt, yt) = keras.datasets.mnist.load_data()
clf = keras.Sequential([layers.Input((28, 28, 1)), layers.Conv2D(16, 3, activation=
    "relu"), layers.MaxPooling2D(),
                        layers.Conv2D(32, 3, activation="relu"), layers.
    MaxPooling2D(), layers.Flatten(),
                        layers.Dense(10, activation="softmax")])
clf.compile("adam", "sparse_categorical_crossentropy", metrics=["accuracy"])
clf.fit((xc / 255.0)[..., None], yc, epochs=3, batch_size=128, verbose=0)
gen = G(tf.random.normal((2000, LATENT), seed=5), training=False).numpy()
p = clf.predict((gen + 1) / 2, verbose=0)
print("Classifier accuracy on real test digits: %.4f" % clf.evaluate((xt / 255.0)
    [..., None], yt, verbose=0)[1])
print("Mean max-class confidence on generated digits: %.3f" % p.max(1).mean())
print("Share of generated digits classified with p>0.9: %.1f%%" % (100 * (p.max(1)
    > 0.9).mean()))
print("Class histogram of generated digits (mode-collapse check):", np.bincount(p.
    argmax(1), minlength=10).tolist())
```

Listing 1. `experiment_code.py`

Output

Imports and configuration

```
TensorFlow 2.21.0 | latent dim 100 | batch 128 | epochs 15
```

Loading and scaling the training data

```
training images: (60000, 28, 28, 1) range [-1.0, 1.0]
steps per epoch: 468
```

Defining the Generator and Discriminator

```
Model: "generator"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| dense (Dense)                   | (None, 6272)           |       627,200 |
+---------------------------------+------------------------+---------------+
| batch_normalization             | (None, 6272)           |        25,088 |
| (BatchNormalization)            |                        |               |
+---------------------------------+------------------------+---------------+
| leaky_re_lu (LeakyReLU)         | (None, 6272)           |             0 |
+---------------------------------+------------------------+---------------+
| reshape (Reshape)               | (None, 7, 7, 128)      |             0 |
+---------------------------------+------------------------+---------------+
| conv2d_transpose                | (None, 14, 14, 64)     |       204,800 |
| (Conv2DTranspose)               |                        |               |
+---------------------------------+------------------------+---------------+
| batch_normalization_1           | (None, 14, 14, 64)     |           256 |
| (BatchNormalization)            |                        |               |
+---------------------------------+------------------------+---------------+
| leaky_re_lu_1 (LeakyReLU)       | (None, 14, 14, 64)     |             0 |
+---------------------------------+------------------------+---------------+
| conv2d_transpose_1              | (None, 28, 28, 32)     |        51,200 |
| (Conv2DTranspose)               |                        |               |
+---------------------------------+------------------------+---------------+
| batch_normalization_2           | (None, 28, 28, 32)     |           128 |
| (BatchNormalization)            |                        |               |
+---------------------------------+------------------------+---------------+
| leaky_re_lu_2 (LeakyReLU)       | (None, 28, 28, 32)     |             0 |
+---------------------------------+------------------------+---------------+
| conv2d (Conv2D)                 | (None, 28, 28, 1)      |           801 |
+---------------------------------+------------------------+---------------+
 Total params: 909,473 (3.47 MB)
 Trainable params: 896,737 (3.42 MB)
 Non-trainable params: 12,736 (49.75 KB)

Model: "discriminator"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| conv2d_1 (Conv2D)               | (None, 14, 14, 32)     |           832 |
+---------------------------------+------------------------+---------------+
| leaky_re_lu_3 (LeakyReLU)       | (None, 14, 14, 32)     |             0 |
+---------------------------------+------------------------+---------------+
| dropout (Dropout)               | (None, 14, 14, 32)     |             0 |
+---------------------------------+------------------------+---------------+
| conv2d_2 (Conv2D)               | (None, 7, 7, 64)       |        51,264 |
+---------------------------------+------------------------+---------------+
| leaky_re_lu_4 (LeakyReLU)       | (None, 7, 7, 64)       |             0 |
+---------------------------------+------------------------+---------------+
| dropout_1 (Dropout)             | (None, 7, 7, 64)       |             0 |
+---------------------------------+------------------------+---------------+
| flatten (Flatten)               | (None, 3136)           |             0 |
+---------------------------------+------------------------+---------------+
| dense_1 (Dense)                 | (None, 1)              |         3,137 |
+---------------------------------+------------------------+---------------+
 Total params: 55,233 (215.75 KB)
 Trainable params: 55,233 (215.75 KB)
 Non-trainable params: 0 (0.00 B)


untrained G(z) shape: (1, 28, 28, 1) | D(G(z)) logit: -0.0010288275079801679
```

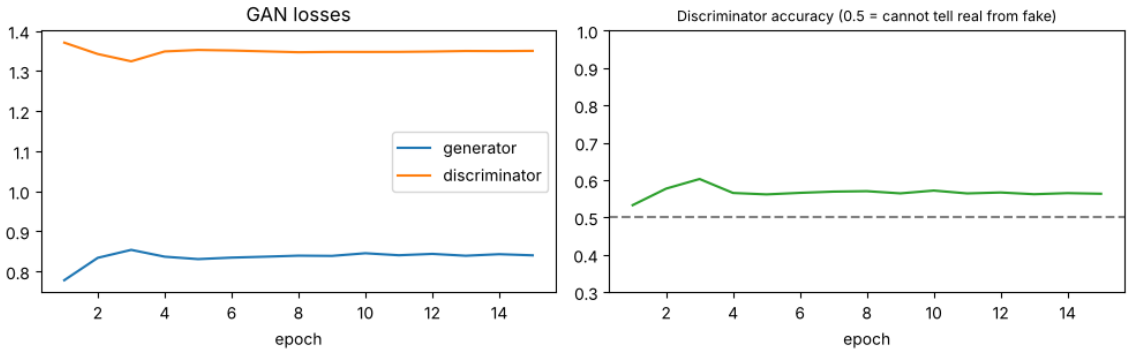
Loss functions, optimisers and the training step

```
train_step compiled with tf.function
```

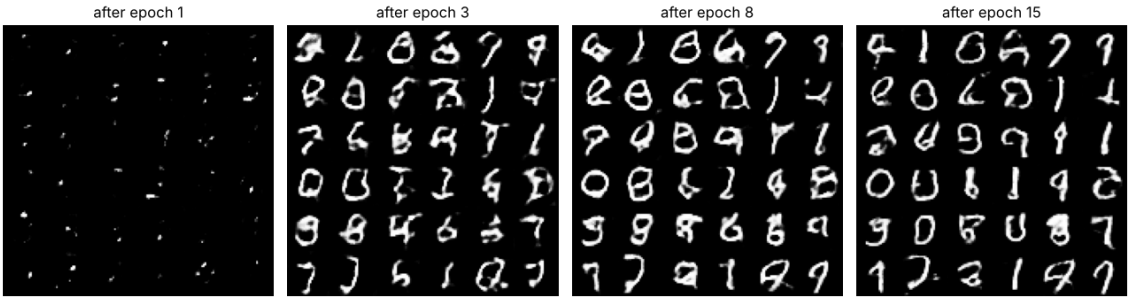
Training the GAN (loss log)

```
epoch  1/15  g_loss 0.778  d_loss 1.371  d_acc 0.53  (105s)
epoch  2/15  g_loss 0.834  d_loss 1.342  d_acc 0.58  (259s)
epoch  3/15  g_loss 0.854  d_loss 1.324  d_acc 0.60  (420s)
epoch  4/15  g_loss 0.837  d_loss 1.349  d_acc 0.57  (587s)
epoch  5/15  g_loss 0.831  d_loss 1.353  d_acc 0.56  (789s)
epoch  6/15  g_loss 0.835  d_loss 1.351  d_acc 0.57  (948s)
epoch  7/15  g_loss 0.837  d_loss 1.349  d_acc 0.57  (1150s)
epoch  8/15  g_loss 0.840  d_loss 1.347  d_acc 0.57  (1352s)
epoch  9/15  g_loss 0.839  d_loss 1.348  d_acc 0.56  (1554s)
epoch 10/15  g_loss 0.846  d_loss 1.348  d_acc 0.57  (1710s)
epoch 11/15  g_loss 0.841  d_loss 1.348  d_acc 0.56  (1852s)
epoch 12/15  g_loss 0.844  d_loss 1.349  d_acc 0.57  (1955s)
epoch 13/15  g_loss 0.839  d_loss 1.350  d_acc 0.56  (2057s)
epoch 14/15  g_loss 0.843  d_loss 1.350  d_acc 0.57  (2158s)
epoch 15/15  g_loss 0.840  d_loss 1.350  d_acc 0.56  (2300s)
```

Training curves



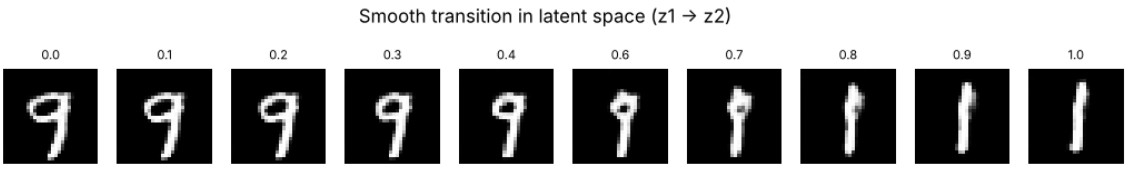
Generated digits during training (same noise every time)



Final samples from the trained generator



Latent-space interpolation between two noise vectors



Quantitative check with an independent MNIST classifier

```
Classifier accuracy on real test digits: 0.9787
Mean max-class confidence on generated digits: 0.863
Share of generated digits classified with p>0.9: 60.3%
Class histogram of generated digits (mode-collapse check): [222, 237, 185, 256, 205, 155, 126,
    233, 155, 226]
```

Conclusion

A DCGAN was implemented and trained on MNIST for 15 epochs. Starting from pure noise, the generator learned to produce digit-like images within a few epochs (see the evolution grid). Training stayed in a balanced adversarial regime: the discriminator accuracy settled at about 0.57 and both losses were stable, with no collapse. Interpolating between two latent vectors produced a smooth transition (a 9 gradually becoming a 1), which indicates a meaningful latent space. An independent classifier (97.9% accurate on real digits) assigned a mean confidence of 0.86 to the

generated digits and was more than 90% sure for 60.3% of them, and all ten classes were generated in comparable numbers (126 to 256 out of 2000), so there was no mode collapse. The remaining blurry or malformed samples show that a small DCGAN trained for a short time on a CPU does not yet match the realism of real digits; longer training would improve this.